

Object-Oriented Programming in Java

Emily Lucia Antosch

17.09.2026

Today's route

1. Organization	2
2. Introduction	11
3. The Java Programming Language	28
4. The First Program	38
5. Literature	46
6. License Notice	48

1. Organization

What you'll know

1. **How the course works**

Lectures, labs, preparation, and the exam.

2. **What you need**

Prior knowledge and the software to install.

3. **How to reach me**

Introductions and contact details.

About me

Emily Lucia Antosch

Backend Software Engineer
at MOIA · 25 years old

B.Sc. Electrical and Information
Technology

M.Sc. in progress
Practical Computer Science

emilylucia.antosch@haw-hamburg.de



Info

I'm new to university teaching. Please be patient with me.

Course schedule

Wednesdays · 15:55–19:15

- The early sessions prepare you for the labs.
- Take part in short questions and code-along exercises.



Memorize

Ask as soon as something is unclear. I'm happy to explain it again.

Laboratory sessions

4

labs this semester

2

people per team



Warning

Successful lab participation is mandatory. Without it, you cannot take the exam.

Prepare before each lab

- There will be detailed instructions coming from the lab lecturers soon!
- You will work through some tasks at home, and some tasks will be done in session.
- If you get stuck, be ready to explain your approach and what the task asks for.
- Lab lecturers will help you finish the parts you could not complete.



Memorize

Bring questions, not blank pages.

Present your work

Present at least two tasks this semester.

- You may not get to choose which tasks you present.
- Be ready to present any task you have worked on.



Info

Start early. Don't leave both presentations until the last lab.

What we'll build on

- Use your prior programming knowledge to write simple Java programs.
- Learn object-oriented programming and compare it with other paradigms.

The module handbook contains the full course description.

Set up your tools

Java SDK

Install the Java development tools.

The setup guide is in the Moodle room.

IntelliJ IDEA

We'll use JetBrains' IDE throughout the course.

I recommend it for beginners.

2. Introduction

What you'll learn

1. **Connect old and new concepts**
Build on your previous programming lectures.
2. **Think in objects**
Understand classes and how this approach differs from C.
3. **Run a Java program**
Create and execute a program in IntelliJ IDEA.

Start with the fundamentals

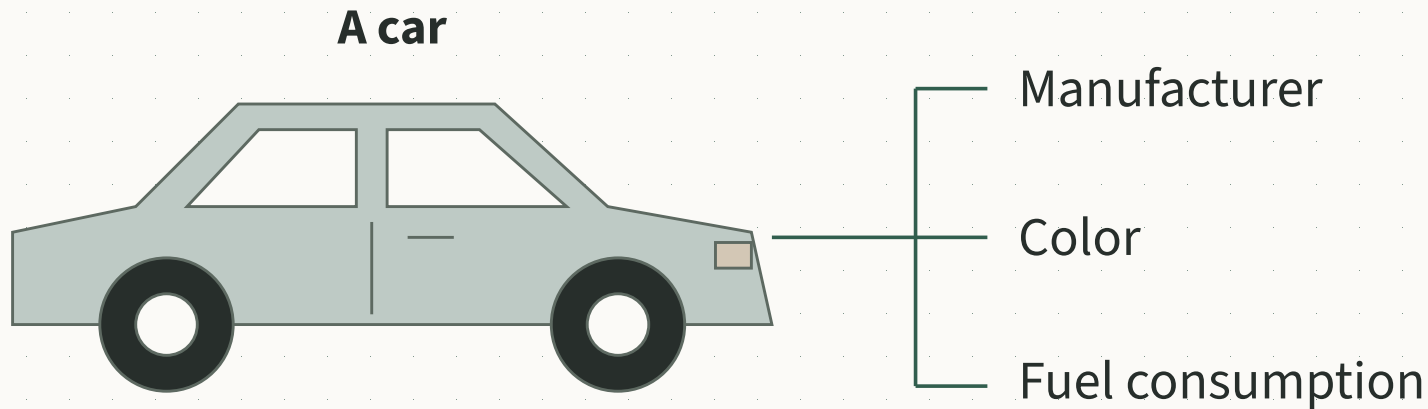
1. **Imperative concepts**
2. **Classes and objects**
3. **Class library**
4. **Inheritance**
5. **Interfaces**

Then extend the toolkit

- 6. **Graphical user interfaces**
- 7. **Exception handling**
- 8. **Input and output**
- 9. **Multithreading** — parallel computing

Objects and classes

We describe things through their properties.



Memorize

Object-oriented programming brings this way of thinking into code.

Objects and classes



Which properties describe a person? Which would a program need?

Objects and classes



Which properties describe a person? Which would a program need?

Student record

Name, address, student ID

Website account

Username, password, join date

Objects and classes

One blueprint. Many objects.

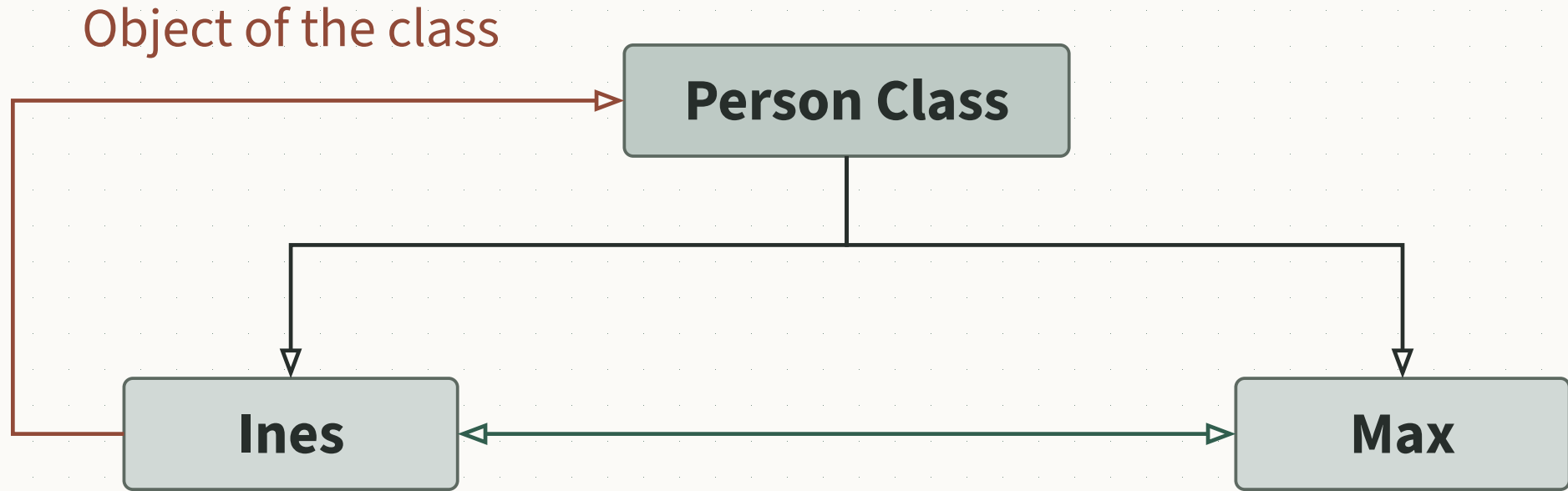
- A **class** defines the properties as variables.
- Each **object** has its own values for those properties.



Info

The **Student** class can describe both **Max** and **Ines**: different names, different student IDs.

Objects and classes



Same class, different object

Figure 1: One class describes multiple objects

Objects and classes

A class brings variables and functions together.

Attributes

Variables that store the object's data.

State

The current values of those attributes.

Methods

Functions that read or change those values.

Objects and classes

UML class diagrams describe attributes and methods.

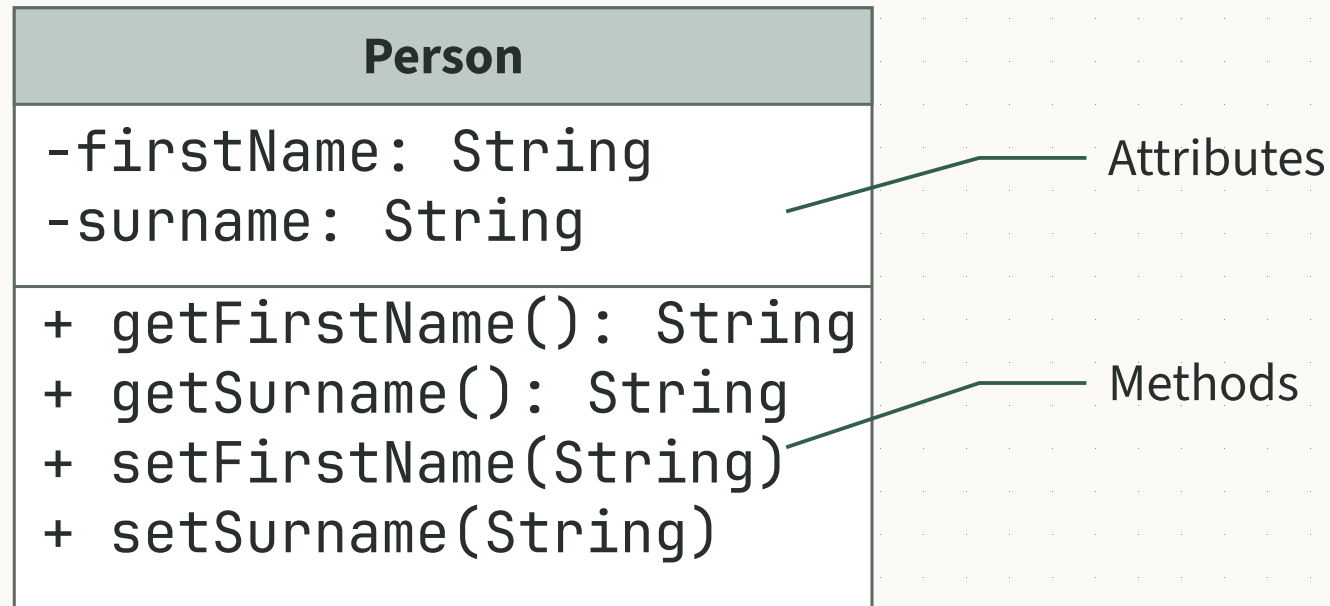


Figure 2: A UML class diagram for Person

Data encapsulation

Methods control access to an object's attributes.

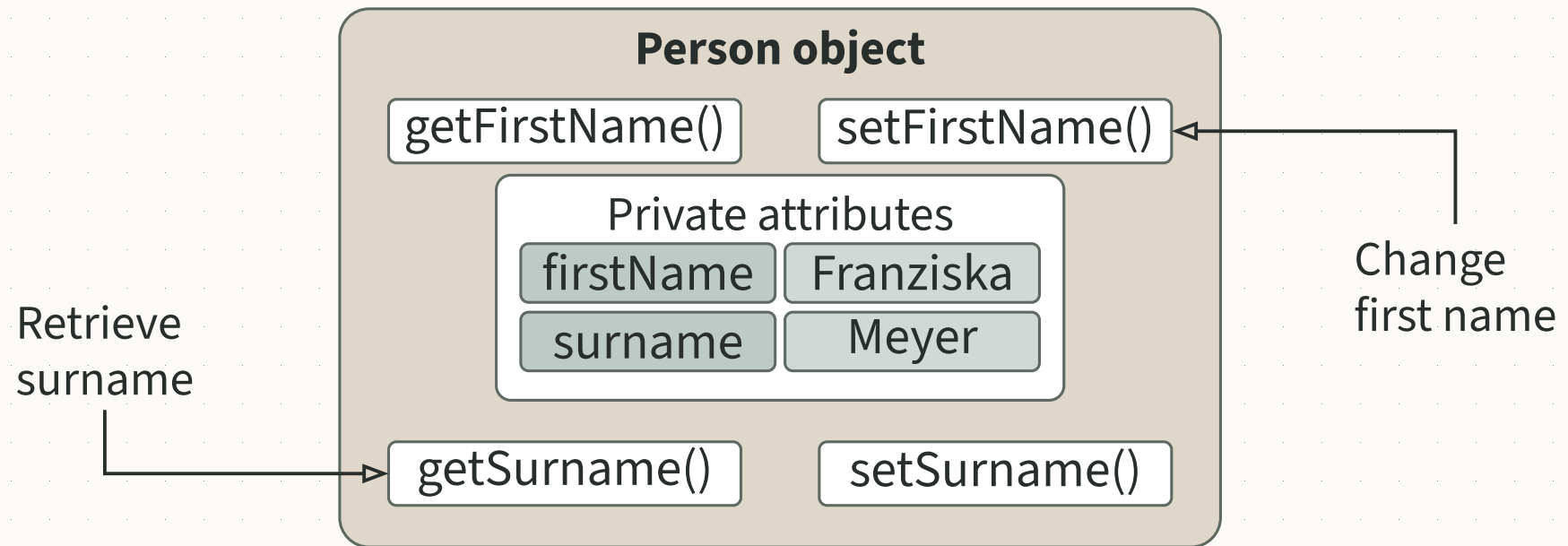


Figure 3: Access data through the class's methods

Data encapsulation

Protect the object's state.

- Encapsulated data is not directly accessible to every part of the program.
- Methods can check values before changing an attribute.
- This prevents invalid state and unintended access.

Data encapsulation



How does this differ from a struct in C?

Data encapsulation

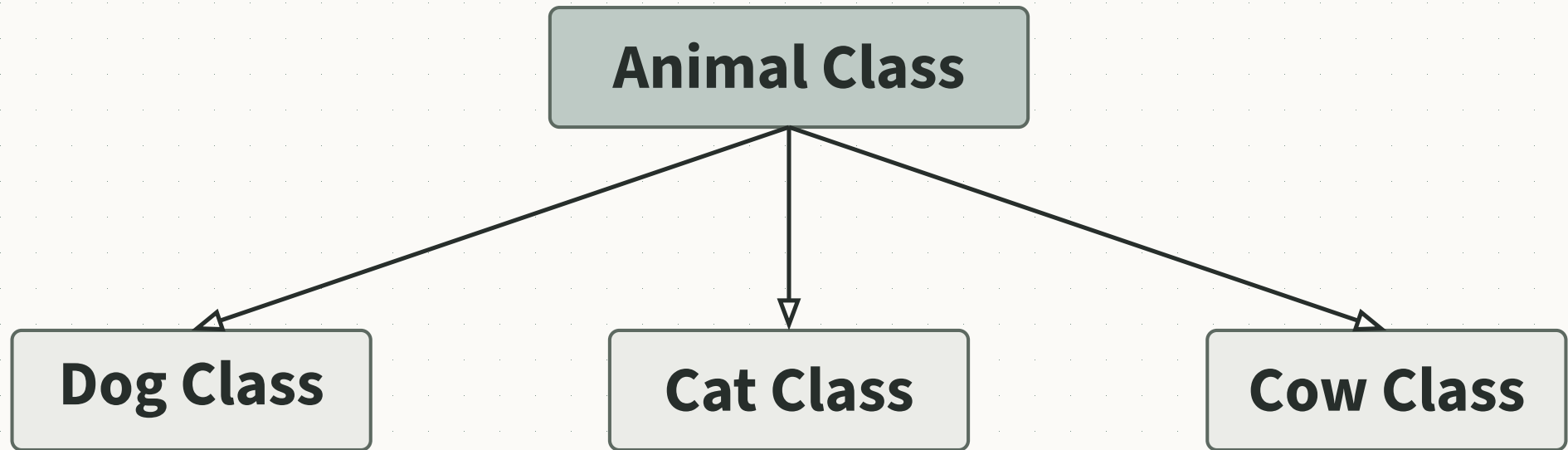


How does this differ from a struct in C?

- Direct field access requires the struct definition to be visible.
- A public struct does not protect its data.
- The data and the functions are not combined into a class.

Inheritance

Derive a new class from a base class: reuse its attributes and methods, then extend it without duplicating code.



Composition

Build classes from other classes: a **House** contains **Windows**, **Walls**, and **Doors**.

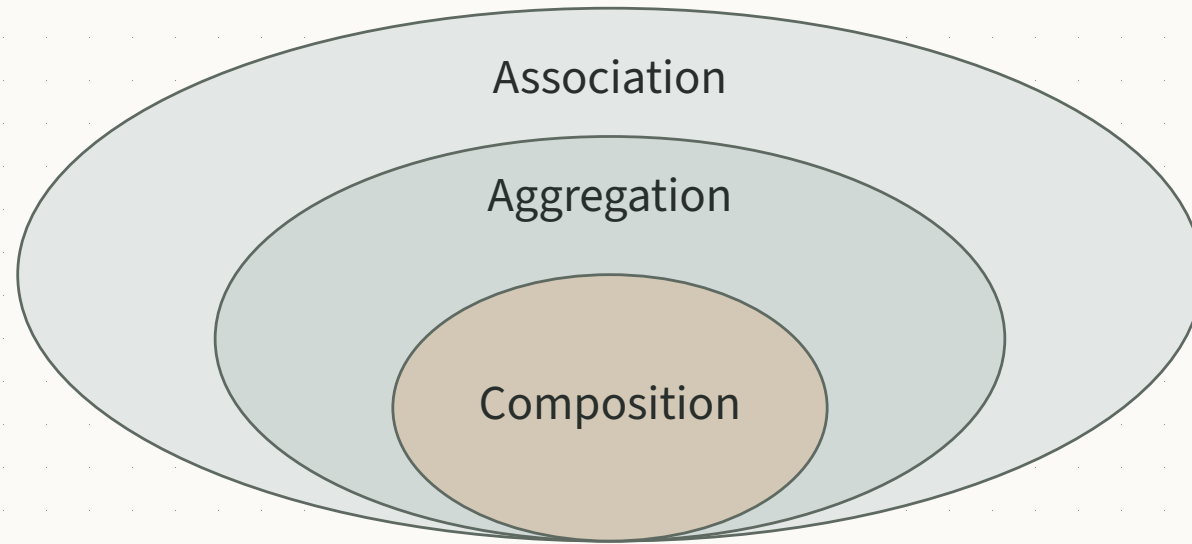
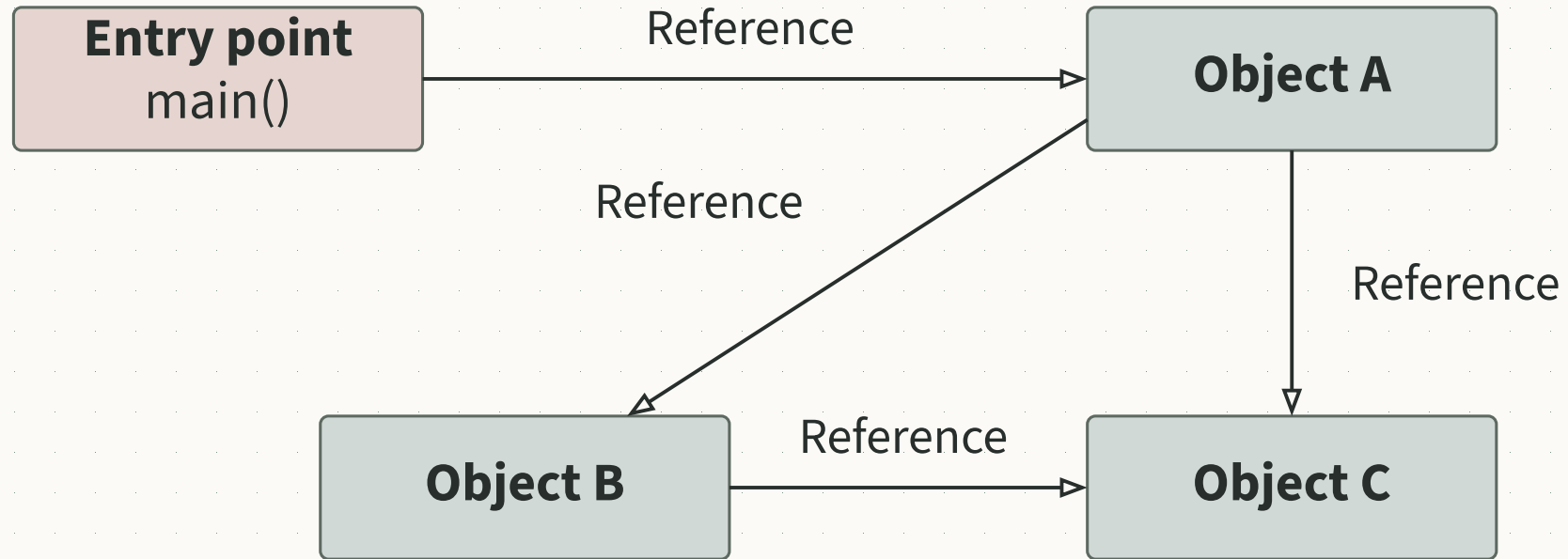


Figure 5: Composition is a strong whole–part relationship

Program execution in Java

1. Execution starts in the static **main** method.
2. It creates objects and stores their **references** in variables.
3. Use those references to access the objects.
4. Objects can create other objects and call their methods.
5. In our single-threaded example, execution ends when **main** returns.

Program execution in Java



Objects created during execution

Figure 6: References connect objects during execution

3. The Java Programming Language

Java vs. C



Task 1

Write these in **C**:

- Sum the numbers 1 to n with a **for** loop.
- Find the larger of two numbers with **if**.
- Put that comparison in a **getMax()** function.

Java vs. C

Java's control flow and syntax will look familiar from C and C++. We'll build on what you already know.

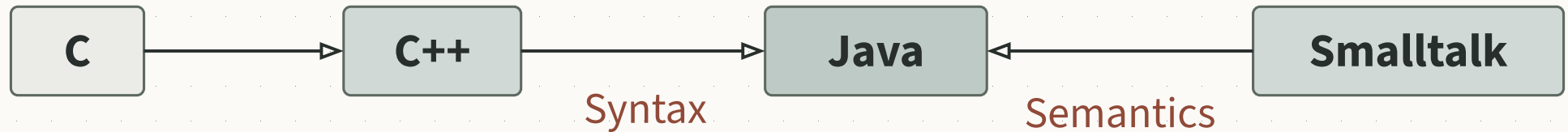


Figure 7: Influences on the Java programming language

Compilation

1. Develop

Write source code on your PC.

The compiler turns it into **bytecode**.

2. Execute

The **JVM** (Java Virtual Machine) runs the bytecode.

No recompilation for each target platform.

Compilation

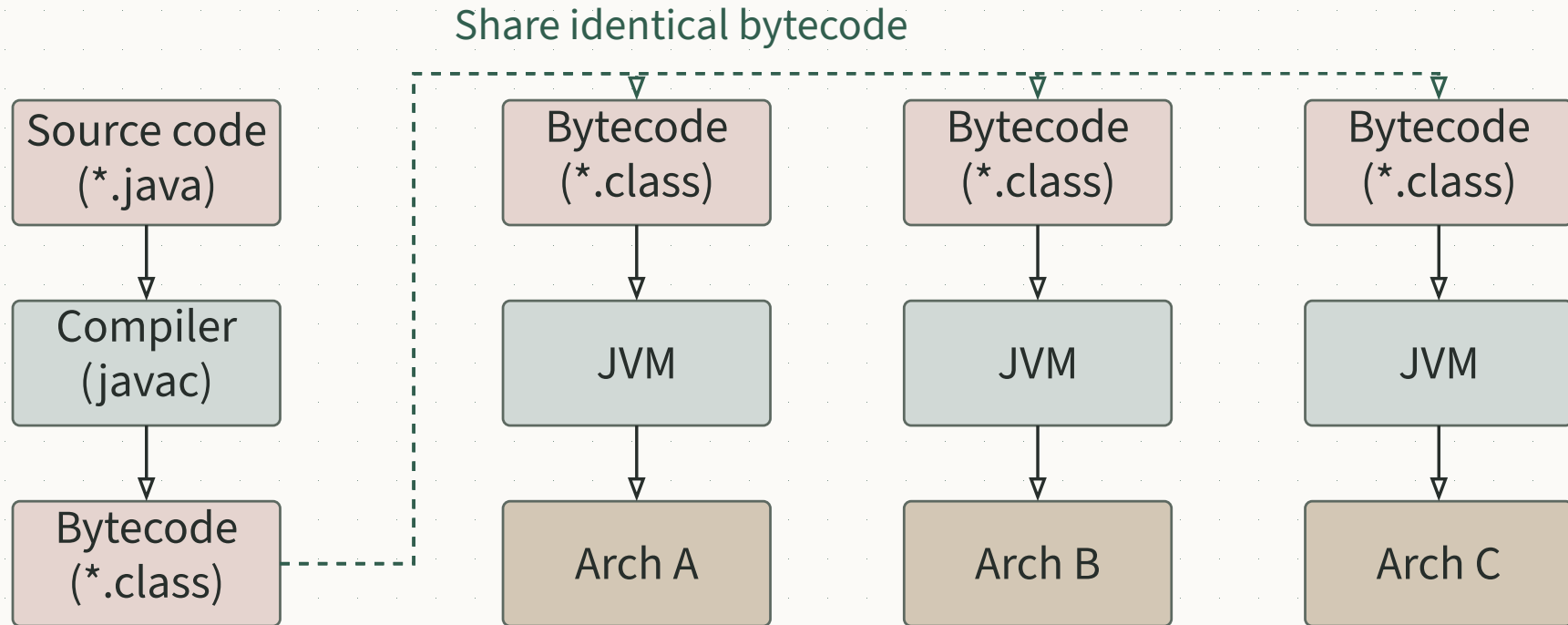


Figure 8: Compile once; run bytecode on each platform's JVM

Compilation

Compiled

Recompile the source for each target platform.

Interpreted

Run the source with an interpreter on the target platform.

Compilation

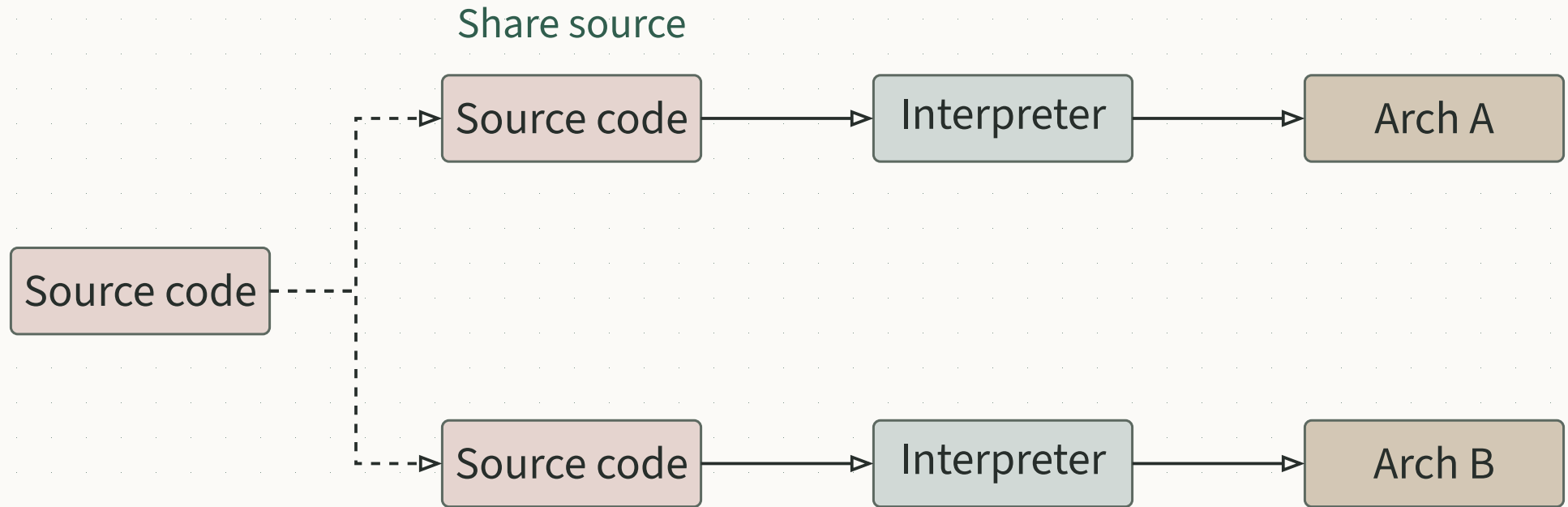


Figure 9: Execution of interpreted languages

Compilation

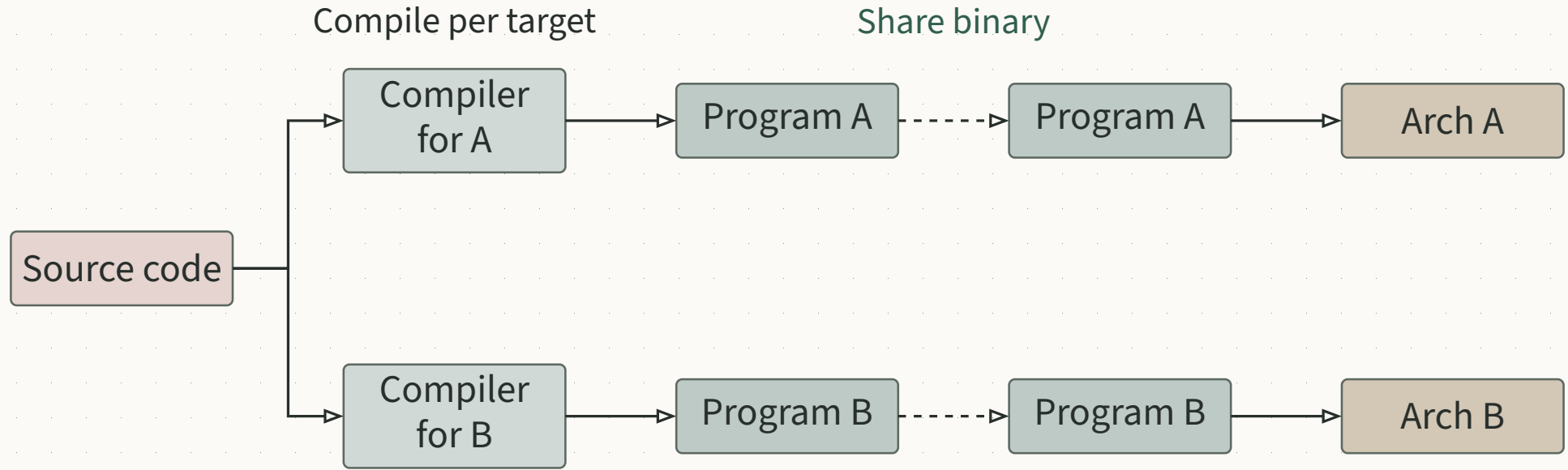


Figure 10: Execution of compiled languages

Properties of Java



What would you want from a new language? What would you change about C/C++?

Properties of Java



What would you want from a new language? What would you change about C/C++?

- **Object-oriented** — classes, objects, inheritance
- **Platform-independent** — the JVM
- **Strongly typed** — types such as `int` and `String`
- **Automatic memory management** — the garbage collector

Properties of Java

Which is the better language?

C or Java?

4. The First Program

Your development environment

IntelliJ IDEA is the IDE we'll use in class and in the exam.

It can set up the Java JDK you need to program.

Download IntelliJ IDEA or scan the code.
Choose the Community Edition.



Your development environment

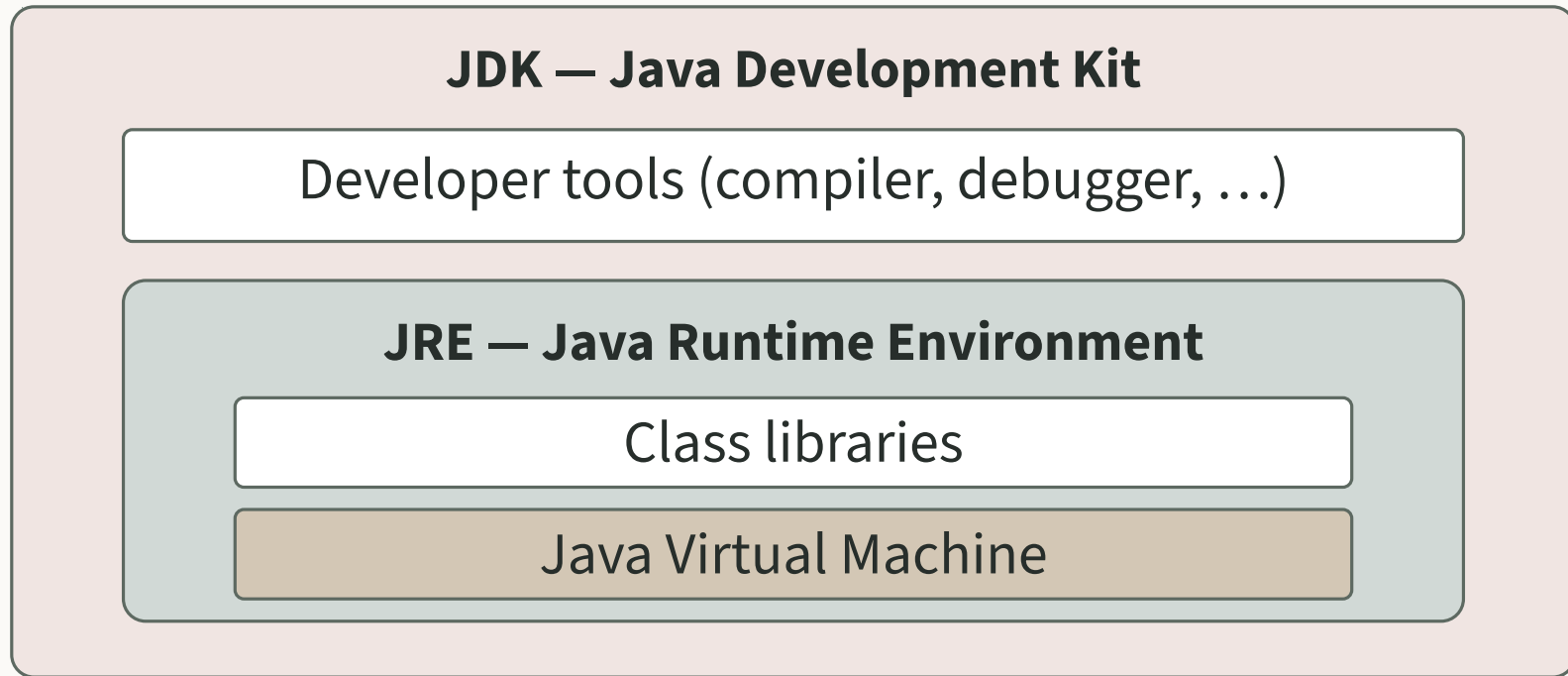


Figure 11: The structure of the Java Toolchain

Your development environment



Task 2

Prepare: choose a directory for your files and open IntelliJ IDEA.

1. Select **File > New > Project**.
2. Give the project a name and location.
3. Choose **Java, IntelliJ**, and the appropriate **JDK**.
4. Click **Create**.

Your development environment



Task 3

Create a package

1. Right-click **src**.
2. Select **New > Package**.
3. Enter a name.

Create a class

1. Right-click the package.
2. Select **New > Java Class**.

The first program

```
1 public static void main(String[] args) {  
2     System.out.println("Hello World!");  
3 }
```

Java



Memorize

Enter this method inside the class you just created. Ready already?
Code along.

Applications in Java

Public class

```
public class MyApplication {...}
```

Matching filename

```
MyApplication.java
```

Entry point

```
public static void main(String[] args)
```

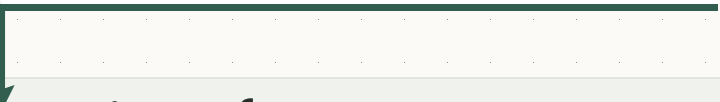
Applications in Java

```
1 public class MyApplication {  
2     public static void main(String[] args) {  
3         System.out.println("Hello World!");  
4     }  
5 }
```

Java

Applications in Java

This name is freely selectable.



```
1 public class MyApplication {  
2     public static void main(String[] args) {  
3         System.out.println("Hello World!");  
4     }  
5 }
```

Java

Applications in Java

This name is freely selectable.

```
1 public class MyApplication {  
2     public static void main(String[] args) {  
3         System.out.println("Hello World!");  
4     }  
5 }
```

Java

This method must always be called main.

5. Literature

Read further

D. Abts

Grundkurs JAVA

Springer-Vieweg

H.-P. Habelitz

Programmieren lernen mit Java

Rheinwerk Computing

6. License Notice

Attribution

CC BY-NC-SA 4.0 and its Public License

<https://creativecommons.org/licenses/by-nc-sa/4.0/>

Based on material by **Prof. Dr. Marc Hensel**.
Some images, text, and the layout have been changed.

The source material was supplied privately. Its link cannot be shared with the audience.